SafeHer

Smart Women Safety & Emergency Response System

Complete Final Year Project Specification

For implementation using Codex, Flutter, Firebase and Android native Kotlin

Project type	Android mobile application (FYP)
App name	SafeHer
Suggested tagline	Your Safety, Our Priority.
Primary stack	Flutter + Dart, Firebase, Google Maps, WebRTC, Kotlin Android integration
Intended users	Users who voluntarily connect with trusted emergency contacts

This document translates the project requirements into an implementation plan for a coding agent. It defines the features, architecture, data model, permissions, safety constraints, implementation phases and acceptance tests.

Critical engineering note: Android restricts background camera/microphone use and third-party apps cannot universally intercept power-button sequences. The app must not promise features that the operating system or device cannot support. Core SOS and location features should remain useful even when media or hardware shortcuts are unavailable.

1. Project Overview

SafeHer is an Android safety application built with Flutter. Users sign in with Google, find other registered users, send and accept friend requests, and explicitly designate trusted emergency contacts. When a user activates SOS, the app creates an emergency event, notifies their selected contacts, shares current location updates, and offers navigation to the location. Where device, OS permissions and user consent allow, the system can also establish a live audio/video session.

Problem statement

In an emergency, a person may not have time to open an app, type a message, explain their location, or call each contact individually. SafeHer aims to reduce these steps by providing a fast SOS workflow and sharing useful, permission-controlled information with pre-approved contacts.

Main objectives

- Provide a simple, accessible SOS interface.
- Allow users to authenticate securely with Google.
- Create a consent-based trusted-contact network using friend requests and explicit emergency-contact confirmation.
- Deliver SOS notifications to selected contacts and record the emergency event.
- Share the sender's location with timestamp and accuracy, updating it while an emergency is active when permitted.
- Open the sender's location in Google Maps or another installed navigation app.
- Support an opt-in live audio/video session with a trusted contact, subject to Android permission and background-execution rules.
- Explore configurable hardware/gesture shortcuts without claiming unsupported system-level controls.
- Provide cancellation, false-alarm handling, emergency history, privacy controls and secure access rules.

2. Functional Requirements

2.1 Google authentication and user profile

- Display a welcome screen with 'Continue with Google'.
- Use Firebase Authentication for Google sign-in; do not store Google passwords.
- Create/update a Firestore user profile containing UID, display name, email, profile image URL, createdAt and user-selected settings.
- Provide sign-out and account deletion flows, with clear explanation of data deletion.
- Do not expose private profile fields to arbitrary users.

2.2 Search, friend requests and trusted contacts

- Users can search by a deliberately chosen unique username or invitation code. Avoid exposing a full directory of personal email addresses.
- A user can send, accept, reject, cancel or remove a friend request.
- Only accepted friends can request to become trusted emergency contacts.
- Trusted-contact status requires explicit confirmation by both people; friendship alone must not silently enable location or media access.
- Show contact status clearly: Pending, Friend, Trusted Contact, Request Declined or Removed.
- Allow either user to revoke trusted-contact status at any time.
- Provide a test/demo flow for FYP evaluation without using real emergency services.

2.3 SOS activation

- The home screen has a prominent SOS button with clear touch target and accessibility label.
- A user can activate SOS from the app UI; optional short countdown/cancel window may reduce accidental alerts, but the user must be able to configure it.
- When activated, create a unique emergencyId and store sender UID, event status, createdAt, trigger source, initial location if available and the authorized recipients.
- Notify every currently active trusted emergency contact.
- Show the sender a visible state such as SOS Active, Sending Alert, Location Sharing, Limited Connectivity or Cancelled.
- Prevent duplicate accidental events through idempotency and debounce rules; a deliberate repeated trigger should not silently disappear.
- Allow the sender to cancel/resolve an SOS and notify recipients that the emergency was cancelled or resolved. Keep a timestamped audit record.
- If internet is unavailable, show the actual limitation, retain a local pending state where practical and retry safely when connectivity returns. Never claim an alert was delivered unless the server confirms processing.

2.4 Recipient alert and alarm

- Use Firebase Cloud Messaging to send push notifications to registered devices belonging to authorized recipients.
- Notification payload should contain minimal information: emergencyId, sender display name and event type. Fetch protected details after authentication and authorization.
- Open an emergency detail screen when the recipient taps the notification.
- Implement Android notification channel, vibration and sound settings; request notification permission where required.
- A full-screen/high-priority alarm-like experience may be attempted only where Android permits it and the use case/permissions qualify. Do not bypass Do Not Disturb, device security or user settings.
- If the recipient's app is closed, delivery depends on network, OS state, device power settings and notification permissions. The UI must not promise guaranteed delivery.

2.5 Real-time location and Google Maps

- Request location permission contextually and explain why location is shared.
- Share location only during an active emergency or another clearly enabled, consented session.
- Location updates should include latitude, longitude, timestamp and horizontal accuracy; show 'last updated' and stale-location warnings.
- The recipient sees a map marker, sender name, update time, accuracy where available and a button to navigate.
- Use a Google Maps URL/intent or Maps SDK to open directions to the latest valid coordinates.
- Handle permission denial, GPS disabled, poor accuracy, stale updates, offline mode and location service failure.
- Stop live sharing when SOS is resolved/cancelled, unless the user explicitly chooses a separately explained follow-up sharing period.

2.6 Live audio/video and camera

- Use WebRTC for a peer-to-peer or relayed audio/video session; Firebase may be used for signaling, but it is not itself a live video transport service.
- An SOS may create an 'Emergency media request' for selected, authorized contacts. Make the request status visible to the sender.
- Camera and microphone activation must follow Android permissions, foreground-service requirements and applicable platform policies.

- Use visible indicators and a clear session state. Do not implement covert recording, hidden camera activation, or bypass OS privacy indicators.
- Default design: show an explicit on-device consent/setup screen for camera/microphone and make the session visible. If the app is backgrounded/locked and the OS prevents camera/mic startup, keep SOS/location functional and show that media is unavailable.
- Use authenticated signaling, short-lived session tokens and authorization checks before joining a session.
- Do not save or upload recordings by default. If recording is included, require explicit consent, show a recording indicator, define retention/deletion and restrict access.

2.7 Hardware shortcuts and alternate SOS triggers

- Investigate a configurable shortcut screen for supported trigger types.
- Power-button double press is controlled by Android/device firmware and is not a generally available app-level hook. Do not promise universal interception.
- Volume-button combinations are restricted and may conflict with system controls. Implement only if an allowed, testable Android implementation exists for supported devices.
- A shake gesture or in-app SOS shortcut may be offered as an alternative, with false-trigger protection and user-controlled settings.
- Use native Kotlin/Android integration where needed; document exact supported Android versions and devices.
- Never abuse Accessibility Service, notification access, device-admin privileges or overlay tricks to bypass Android security. Accessibility services should only be used for genuine accessibility functions and with transparent consent.
- Show a capability test in Settings so users know whether their selected shortcut is supported.

2.8 Optional FYP enhancement features

- Emergency history with status and timestamps.
- Battery/network warning while sharing location, where the information is available.
- A safe-journey check-in timer as a later enhancement.
- User report/block controls and rate limits to prevent harassment or false SOS spam.
- Admin/demo dashboard only if the supervisor requires it; restrict access and avoid unnecessary collection of sensitive data.

3. User Flows

Flow A — First-time setup

1. User opens SafeHer and reviews a short privacy explanation.
2. User signs in with Google.
3. User chooses a username and grants only the permissions needed for the feature they are trying to use.
4. User searches for a friend using username/invitation code and sends a request.
5. The other user accepts the request.
6. Both users separately confirm trusted-contact status and review what SOS may share.
7. The app tests notification readiness and explains any missing permission or unsupported device capability.

Flow B — SOS from app

1. Sender taps SOS.
2. App creates an emergency event through an authenticated backend function or secure Firestore transaction.
3. Backend validates sender and active trusted contacts, then sends FCM notifications.
4. Sender's app begins permitted location updates and shows live status.

5. Recipient opens the alert, sees the latest location and can open navigation.
6. Recipient can request/join a permitted live media session; the sender's device follows OS and consent rules.
7. Sender cancels/resolves SOS; backend updates status and informs recipients.

Flow C — SOS via shortcut

1. User configures a shortcut supported by their device.
2. When the shortcut is detected under supported conditions, the app invokes the same SOS pipeline as the in-app button.
3. If the OS blocks the trigger while the app is force-stopped, locked or backgrounded, the app must not claim that it worked.
4. Settings must explain limitations and offer an in-app alternative.

4. Recommended Architecture

Layer	Responsibility	Suggested implementation
Flutter UI	Screens, state, form validation, status display	Flutter + Dart; a maintainable state solution such as Riverpod or Bloc
Authentication	Google sign-in and identity	Firebase Authentication
Application data	Profiles, friend requests, trusted contacts, emergency metadata	Cloud Firestore with strict Security Rules
Trusted server logic	Create SOS, validate recipients, send notifications, state transitions	Callable/HTTPS Cloud Functions; never trust a client to authorize itself
Push notifications	Alert registered recipient devices	Firebase Cloud Messaging
Location	Current location and periodic updates	Geolocator or maintained equivalent; native Android service when required
Maps	Map view and directions	Google Maps SDK/URL intent; protect API keys
Live media	Audio/video stream and session signaling	WebRTC; authenticated signaling via Cloud Functions/Firestore; TURN server may be required
Android-specific features	Background service, permissions, shortcuts	Kotlin platform channels / native Android service
Storage	Optional user profile images or explicitly consented media	Firebase Storage with strict rules and retention controls

Architecture rule: clients may request an action, but the backend must verify the authenticated user, event ownership, trusted-contact relationship and current consent before allowing access. Do not put service-account credentials or private server secrets in the Flutter app.

5. Proposed Firestore Data Model

This is a starting schema. The coding agent may refine it, but must document every change and write matching Security Rules.

Collection/path	Example fields
users/{uid}	uid, displayName, username, email, photoURL, createdAt, settings, notificationTokens (prefer a protected subcollection)
friendRequests/{requestId}	fromUid, toUid, status (pending/accepted/rejected/cancelled), createdAt, updatedAt
users/{uid}/trustedContacts/{contactUid}	contactUid, status, confirmedBySender, confirmedByContact, permissionsSummary, createdAt, updatedAt

Collection/path	Example fields
emergencies/{emergencyId}	senderUid, status, triggerSource, createdAt, updatedAt, resolvedAt, authorizedRecipientUids, lastLocationAt, mediaSessionStatus
emergencies/{emergencyId}/locations/{locationId}	lat, lng, accuracy, timestamp; restrict reads to sender and authorized active recipients
emergencies/{emergencyId}/events/{eventId}	eventType, actorUid, timestamp, minimal metadata
mediaSessions/{sessionId}	emergencyId, senderUid, recipientUid, status, createdAt, expiresAt; no public access to signaling data
users/{uid}/devices/{deviceId}	FCM token, platform, updatedAt; only owner and trusted backend access

Security rules and backend checks

- Deny all access by default; grant only the minimum needed.
- A user may update their own profile but cannot set their own UID to another user's UID.
- Only the intended recipient can accept/reject a friend request addressed to them.
- Friendship does not equal emergency-media consent.
- Only sender and currently authorized trusted contacts may read an active emergency's protected location.
- Only backend code can send notifications to other users or change protected SOS state fields.
- Validate all coordinates, timestamps, event status transitions and list sizes.
- Use App Check where practical and rate-limit SOS creation and friend requests.
- Remove access promptly when a trusted relationship is revoked; define how revocation affects active emergencies and disclose the policy.
- Do not store live location indefinitely. Apply a clear retention period and delete old location points.

6. Screens Required

Screen	Key content
Splash / onboarding	App logo, short explanation, privacy summary
Google sign-in	Continue with Google, error and loading states
Profile setup	Username, display name, profile image optional
Home	Prominent SOS button, trusted-contact count, permissions/connection status
Find friends	Username/invitation code search, request action
Requests	Received/sent friend requests and status
Trusted contacts	List, mutual confirmation, revoke/remove, permissions summary
Emergency active	SOS status, cancel/resolve control, location sharing state
Incoming SOS	Sender, event time, last location update, open map, media request/join
Live map	Map marker, last updated, accuracy/staleness, directions button
Media session	Visible audio/video status, mute/camera controls, leave session
Emergency history	Past events, timestamps, outcome, limited details
Settings	Privacy, permissions, trigger configuration, notification test, sign out, delete account
Help and limitations	Supported triggers, device limitations, emergency-use disclaimer

7. UI and UX Direction

- Use a polished, modern, accessible design suitable for a university FYP demonstration.
- Suggested palette: deep violet/navy with a clear red SOS action and neutral backgrounds. Red should be reserved for emergency states.
- Use large tap targets, readable text, accessible labels, high contrast and clear loading/error states.
- The SOS button must be visually obvious but protected against accidental taps with a user-configurable short countdown or confirmation method.
- Show real system status rather than fake success: Alert queued, Alert processed, Location unavailable, Media not available, SOS resolved.
- Do not rely on color alone to communicate status.
- Provide a demo mode or test notification feature clearly labelled as a test; never send test alerts disguised as real emergencies.

8. Permissions and Platform Limitations

Capability	Requirements / constraints
Location	Runtime permission; background location needs extra justification and permission. Must explain ongoing sharing and stop when appropriate.
Notifications	Notification permission on Android versions that require it; user may disable notifications.
Camera/microphone	Runtime permissions; visible, OS-compliant use. Background startup may be blocked; no covert activation.
Foreground service	Correct service type and permissions for location/camera/microphone as applicable; follow target SDK rules and Play policies.
Power button	No universal third-party interception API; often reserved for system/device features.
Volume buttons	May be unavailable in background or conflict with system; must be tested and not bypass OS protections.
FCM delivery	Depends on network, device state, app configuration and notification settings; cannot guarantee delivery.
WebRTC	Signaling plus STUN/TURN configuration; NAT/firewall may prevent direct peer connection without TURN.
Maps	API key and billing/configuration may be needed for embedded Maps SDK; a Maps URL intent can reduce setup complexity.

Do not claim that the app can secretly switch on both cameras and microphone on every phone the instant SOS is triggered. Implement the closest legal and technically supported experience, make limitations visible, and ensure SOS notifications/location are not dependent on media permissions.

9. Privacy, Consent and Abuse Prevention

- Explain in plain language what an emergency contact can see and when.
- Use explicit, revocable consent for trusted-contact status and media sessions.
- Do not enable continuous tracking by default.
- Only share location during an active SOS or a separately enabled session.
- Allow users to block/remove contacts and report abusive behavior.
- Use backend authorization on every sensitive read/action; do not rely only on hiding buttons in Flutter.
- Avoid logging raw coordinates, emails, FCM tokens, session credentials or media contents in analytics/debug logs.

- Define retention and deletion for location history and emergency metadata.
- Make camera/microphone use visible and do not record by default.
- For a real deployment, review local laws, Firebase privacy terms, Play Store policy and consent requirements with the project supervisor.

10. Development Phases for Codex

Phase	Deliverable	Definition of done
0. Audit/setup	Inspect repository, toolchain, package IDs and existing code	Written plan; build runs before changes; no blind overwrite
1. UI foundation	Navigation, theme, onboarding, home, settings, placeholders	App builds and core screens are usable
2. Auth/profile	Google sign-in, profile creation, sign-out	Tested success, cancellation and failure paths
3. Friends/consent	Search, friend requests, trusted-contact confirmation	Rules prevent unauthorized writes/reads
4. SOS backend	Emergency state, Cloud Function, FCM	Only authorized contacts notified; idempotent state transitions
5. Location/maps	Location updates, map view, navigation	Permission denial, stale GPS and offline paths tested
6. Live media	WebRTC signaling, join/leave, visible session UI	Only authorized participants can join; no hidden capture
7. Native trigger research	Kotlin integration and supported shortcut prototype	Support matrix and real-device tests documented
8. QA/security	Automated/manual tests, rules tests, error handling	Acceptance checklist passes; known limitations documented
9. FYP delivery	README, architecture diagrams, setup instructions, demo guide	A new developer can configure and run the project

Working method

1. First inspect the current repository and identify whether it is empty or contains an existing Flutter app.
2. Before coding, provide a concise plan, dependencies, Firebase setup needs and platform limitations.
3. Implement in small phases. After each phase, run formatting, static analysis and available tests.
4. Do not replace files wholesale when a focused change is safer.
5. If credentials, Firebase project IDs, Google OAuth configuration, Maps keys or TURN credentials are missing, add placeholders and explain how the owner supplies them securely.
6. Never invent that a feature is tested. Report exact commands run and any blockers.
7. Keep the app buildable throughout; if an advanced feature is blocked, provide a safe interface and document the blocker rather than breaking core SOS.

11. Acceptance Test Checklist

- ■ A user can sign in with Google and sign out.
- ■ User profile is created once and updated without creating duplicates.
- ■ A user can send a friend request; the recipient can accept/reject it.
- ■ A friend is not automatically a trusted emergency contact.
- ■ Trusted-contact status requires the intended confirmation flow and can be revoked.

■ An SOS creates exactly one event for one activation and sends alerts to authorized contacts.

■ An unauthorized user cannot read emergency details or location data.

■ The recipient sees the latest valid location and its update timestamp.

■ The directions action opens navigation to the correct coordinates.

■ Location denial, GPS off, no internet and stale data show honest error states.

■ SOS cancellation/resolution updates the sender and recipients.

■ Push notification sound/vibration works on supported test devices with permissions enabled.

■ The app does not claim guaranteed alerts when delivery is unconfirmed.

■ Camera/microphone are never activated covertly; permissions and session state are visible.

■ Only authorized participants can join a WebRTC session.

■ Media unavailable does not prevent SOS creation or location sharing.

■ Unsupported power/volume shortcuts are clearly labelled and do not falsely report success.

■ Account deletion and data retention behavior are documented.

■ Firestore rules tests cover unauthorized reads/writes.

■ The project builds on a documented Android SDK/Flutter version.

12. FYP Deliverables

• Flutter Android source code and documented project structure.

• Firebase configuration/setup guide with placeholders for secrets.

• Firestore schema and Security Rules.

• Cloud Functions source code and deployment instructions.

• Android native Kotlin code for any supported platform integrations.

• Architecture diagram and user-flow diagram.

• Database schema documentation.

• Test cases and results.

• Project synopsis, objectives, scope, methodology and limitations.

• Demo script for supervisor evaluation.

• Known limitations and future work.

13. Copy-Paste Master Prompt for Codex

Use the following as the primary instruction to Codex after opening the project repository. Codex should read this PDF/specification and implement the app in phases rather than attempting an unverified one-shot build.

```
You are the senior Flutter + Firebase + Android engineer responsible for implementing SafeHer, an
Android FYP app described in the accompanying
SafeHer_FYP_Complete_Project_Specification_for_Codex.pdf.

MANDATORY WORK RULES
1. Inspect the existing repository before editing. Identify Flutter/Dart versions, current structure,
Android package ID, existing code and build status. Do not blindly overwrite working files.
2. First return a concise repository audit, architecture plan, dependency list, Firebase setup
requirements, platform limitations and phased implementation checklist. Then begin Phase 1.
3. Use Flutter/Dart for UI and app logic; Firebase Authentication for Google sign-in; Cloud Firestore
for profiles, friend requests, mutual trusted-contact consent and emergency metadata; Cloud Functions
for privileged SOS processing and FCM notifications; FCM for push alerts; Google Maps/Maps intents
for navigation; WebRTC for consent-based live audio/video; Kotlin/platform channels only where
Android-specific functionality is necessary.
4. Implement strict Firestore Security Rules and backend authorization. Never trust client-side UI
```

```
checks for sensitive access. Never place service-account keys or private secrets in the client.
5. Core flow: Google sign-in -> profile -> friend request -> accept -> separately confirm trusted
emergency contact -> SOS -> notify trusted contacts -> live location while permitted ->
map/directions -> optional authorized, visible audio/video session -> cancel/resolve and notify
contacts.
6. Treat consent as explicit, granular and revocable. Friendship alone must not grant location or
camera/microphone access. Do not build covert recording or hidden camera/microphone activation.
7. Android does not provide a universal third-party power-button double-press hook. Volume shortcuts
and background execution are device/OS restricted. Research and implement only supported,
policy-compliant triggers, document the tested device/version matrix, and never falsely claim
unsupported shortcuts work. Use an in-app SOS as the dependable primary path.
8. Android may prevent camera/microphone use from background or lock screen. Keep SOS creation,
notifications and location independent from media. If media cannot start, show a clear limitation and
preserve the core emergency workflow.
9. Build and test incrementally. After each phase run dart format, flutter analyze, unit/widget tests
and Android build where possible. Fix errors before moving on. Do not claim tests passed unless
actually run.
10. Add loading, empty, permission-denied, network-error, stale-location, notification-disabled and
media-unavailable states. Protect against duplicate SOS events and rate-limit abusive requests.
11. Do not fabricate Firebase project credentials, Maps keys, OAuth setup or TURN credentials. Use
documented placeholders and explain exact setup steps securely.
12. Deliver clean code, comments for complex native integrations, README, Firebase setup, schema,
security rules, Cloud Functions, test plan, known limitations and FYP demo instructions.

Start by auditing the repository. Do not begin with the advanced media or hardware-shortcut features.
Build a stable, secure, working core first: app shell, authentication, profiles, friend requests,
trusted-contact consent, SOS backend, FCM alerts, location and map navigation. Then implement WebRTC
and research supported Android shortcuts. Follow the PDF's requirements and acceptance checklist.
```

14. Important Setup Items the Project Owner Must Provide

- A Firebase project with Authentication, Firestore, Cloud Messaging and Cloud Functions configured.
- Android SHA fingerprints and Google sign-in configuration.
- Google Maps API key only if using the embedded Maps SDK; restrict the key appropriately.
- A secure TURN server/provider configuration if WebRTC needs relay support.
- A physical Android test phone and a second phone/account for recipient testing.
- A test plan and supervisor approval for the emergency-media consent model.

15. Final Recommendation

Implement SafeHer as a reliable consent-based emergency coordination app, not as a promise that every phone can be remotely controlled. The most important FYP success criteria are secure trusted-contact relationships, dependable SOS event processing, honest notification status, real-time location with clear freshness indicators, and working navigation. Live audio/video and hardware triggers are advanced add-ons and should be tested separately.

Suggested FYP title: “SafeHer: A Consent-Based Android Women Safety and Emergency Response System Using Flutter and Firebase.”